When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

1. Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

2. Design/Methodology/Approach

This is a design science study. We read the code of an existing risk management application, ran it, and recorded seven distinct ways in which it failed. Each failure was traced to theory that already explains it, from integrity models, fail-safe defaults and complete mediation to separation of duties, tamper-evident logging and the critique of audit as ritual, and a design principle was drawn from that theory. We built a new system, ERMA-GRC, on these principles. The old and new systems then faced 23 scenarios over the same stored data, under a protocol time-stamped in the project repository beforehand. Eighteen scenarios come from the audit, four go beyond it, and one was meant to fail. All deviations are reported.

3. Findings

Of 18 use cases identified from the audit, the original system prevented the failure in 4. ERMA-GRC stopped it in all 18. It rejected writes while not in a session, would not allow anybody to approve their own work, stamped all the audit lines with the signed-in user rather than a claimed name, picked up an edited line, and displayed a database outage as an error. On top of the audit, it blocked 3 of 4 failures, including two approvers choosing the same record simultaneously. Stored results were retrieved from the database surrogate, never from what a system asserted. The new design was unsuccessful twice, and we report both. Client: Something stored by the client itself would be able to set any state on the server without the server right. One fundamental risk indicator indicated the right state of the screen but the calling client could change anything it wanted to store itself. And one with the audit key could have altered the trail off line, undetectable after a reboot. They also rely on one another: a separation of duties is only as reliable as the audit trail it reads, and a rule checked only in the browser is no guarantee.

This Abstract is submitted to LEAD-2027 conference conducted at IIM Shillong during 20-22 January 2027

4. Originality/Value

We treat the integrity of assurance evidence as something a GRC system's design can protect or destroy, rather than as an organisational outcome. This paper provides a taxonomy of failures observed in a working system, seven principles together with two testable claims towards a design theory, and a scenario protocol that others can rerun. Use the principles as a checklist by an auditor and/or buyer. The two failures defined the next set of tasks: off-system anchoring of audit trails, performing all metrics calculations on the server, and demonstrating the ideas to practitioners.

Keywords

Assurance integrity and GRC information systems: A design science research view on enterprise risk management implementation with tamper-evident audit trail and separation of duties.